

ABSTRACT:

Escaping the Haunted Mansion is a console-based text adventure game developed in C as a CSE115 group project. The game places the player inside a six-room haunted mansion and requires exploration, item collection, inventory management, puzzle solving, health management, and movement between connected rooms. The player begins in the Entrance Hall and must collect the Flashlight, obtain the Silver Key by solving the Library puzzle, reach the Secret Chamber, obtain the Master Key by solving the second puzzle, and return to the Entrance Hall. Ghost encounters in the Kitchen and Basement reduce the player's health, while the Med Kit and Medicine provide ways for the player to recover health. The program applies structures, arrays, strings, functions, parameters, pointers, loops, conditional statements, switch-case selection, input validation, and random number generation.

INTRODUCTION:

Text-based adventure games provide an interactive way of applying programming logic through a sequence of user-controlled decisions. Instead of relying on graphical elements, these games use text, menus, and keyboard input to represent the environment and respond to the player's actions. This makes the genre particularly suitable for demonstrating fundamental programming concepts because the relationship between the program's logic and its output can be observed directly.

Escaping the Haunted Mansion was developed as a CSE115 group project to apply the programming concepts learned throughout the course in a single interactive application. The game places the player inside a six-room haunted mansion consisting of the Entrance Hall, Living Room, Kitchen, Library, Basement, and Secret Chamber. The player must explore the connected rooms, collect useful items, solve puzzles, manage health, and make appropriate decisions to progress through the game. The mansion setting was selected to provide a clear connection between these gameplay elements and the programming logic required to implement them.

The primary objective of the project was to develop a functional console-based game while demonstrating the practical use of C programming concepts. The game incorporates structures, arrays, strings, functions, parameters, pointers, loops, conditional statements, switch-case selection, input validation, and random number generation. These concepts are used throughout the program to represent the player and rooms, control movement, manage the inventory, process puzzles, handle health, and determine the outcome of the game.

The project was developed in stages, beginning with the basic program structure and data organization and progressing toward the complete gameplay system. The final program integrates room navigation, item collection, inventory management, puzzles, ghost encounters, healing, and winning and Game Over conditions into one application. The project therefore demonstrates how the fundamental concepts of CSE115 can be combined to develop a structured and interactive program with a clear objective and progression.

FEATURES:

The game combines several features to create a structured and interactive gameplay experience. Each feature contributes to the player's progression and is connected to the overall objective of escaping the mansion.

I. Feature-1 Mansion Exploration and Movement

The game contains six connected rooms: the Entrance Hall, Living Room, Kitchen, Library, Basement, and Secret Chamber. The player can move between rooms using four directional choices: North, South, East, and West. Available exits are stored for each room, allowing the movement system to determine whether a selected direction is possible.

II. Feature-2 Item Collection and Inventory

Items can be collected from different rooms and stored in the player's inventory. Important progression items include the Flashlight, Silver Key, and Master Key, while the Medicine and Med Kit provide health recovery. The inventory system allows the game to check whether the player has the items required to enter restricted areas or complete the escape.

III. Feature-3 Puzzles and Progression

Two puzzles are included to make progression dependent on the player's actions. Solving the Library puzzle correctly provides the Silver Key, while solving the Secret Chamber puzzle provides the Master Key. The keys are then used to access restricted areas and complete the final objective.

IV. Feature-4 Ghost Encounters and Health

Ghost encounters occur in the Kitchen and Basement and reduce the player's health. The damage is determined using random number generation, making each encounter less predictable. The player must therefore manage health carefully while exploring the mansion.

V. Feature-5 Healing System

The player can recover health by using Medicine or a Med Kit. Medicine restores 20 health, while the Med Kit restores 35 health. The player's health cannot exceed the maximum value of 100, and a healing item is removed from the inventory after it is used.

VI. Feature-6 Score System

The game keeps track of the player's score throughout the adventure. Successfully solving the Library puzzle adds 20 points, while solving the Secret Chamber puzzle adds 30 points. The final score is displayed when the player wins or when the game ends because the player's health reaches zero.

VII. Feature-7 Gameplay Flow

The gameplay follows a structured progression in which each completed task leads to the next stage of the adventure. The player begins in the Entrance Hall, explores the available rooms, collects the Flashlight, solves the Library puzzle to obtain the Silver Key, and uses the required items to access restricted areas. After reaching the Secret Chamber and solving the second puzzle, the player obtains the Master Key and must return to the Entrance Hall to escape. During this progression, ghost encounters can reduce health, making the use of healing items an important part of managing the player's progress.

VIII. Feature-8 Input Validation

Input validation is used to prevent invalid entries from disrupting the gameplay. The program checks whether the player's menu selection is successfully read as an integer. When invalid input is entered, the remaining characters are cleared from the input buffer before the program continues. This allows the game to handle incorrect user input without terminating unexpectedly.

IX. Feature-9 Winning and Game Over Conditions

The game has two possible outcomes. The player wins by obtaining the Master Key and returning to the Entrance Hall. If the player's health reaches zero or below, the game ends with a Game Over message. The final outcome displays relevant information such as the player's name, score, and remaining health.

CSE115 CONCEPTS USED:

The project applies the major C programming concepts covered in CSE115 to different parts of the game. Each concept serves a specific purpose in controlling the gameplay, organizing information, and responding to the player's actions.

CSE115 Concept	Use in the Project
Structures	The <code>Room</code> and <code>Player</code> structures organize related information. <code>Room</code> stores room names, descriptions, exits, items, and puzzle information, while <code>Player</code> stores the player's name, health, score, current room, and inventory.

Arrays	Arrays are used to store room exits and the player's inventory, allowing several related values to be managed within the program.
Strings	Character arrays are used for player names, room names, descriptions, item names, and other text displayed during gameplay.
Functions	The program is divided into functions for tasks such as displaying rooms, moving between rooms, collecting items, checking the inventory, solving puzzles, and managing health.
Function Parameters	Parameters allow functions to receive specific player, room, item, or game information when performing different operations.
Pointers	Pointers allow functions to access and modify shared player or game data directly when changes need to be reflected in the main program.
Loops	Loops repeatedly process menus, player actions, inventory searches, and gameplay until a winning or Game Over condition is reached.
Conditional Statements	<code>if</code> , <code>else if</code> , and <code>else</code> statements control movement restrictions, item requirements, puzzle results, health conditions, and other game rules.
Switch-Case Selection	Switch-case statements process menu and movement choices and direct the program to the appropriate action selected by the player.
Input Validation	User input is checked before being processed so that invalid choices can be handled without incorrectly changing the game state or terminating the program.
Random Number Generation	Random numbers are used during ghost encounters to determine damage, making the health loss from encounters less predictable.

INDIVIDUAL MEMBER CONTRIBUTION:

The project was divided into three assigned source-code sections so that each member could develop and explain a distinct part of the program. The sections were integrated through the common `Room` and `Player` structures and the shared game state, allowing the individual contributions to function together as one complete application.

1. Syed Mohammad Aqif Reza | Student ID: 2622644042

Aqif was responsible for establishing the foundation of the program. His assigned section includes the header files and constants, `Room` and `Player` structures, function prototypes, main menu, room initialization, player initialization, and room display. The `Room` structure organizes information such as room names, descriptions, exits, items, puzzles, and ghost conditions, while the `Player` structure stores the player's name, health, score, location, and inventory. The room and player initialization functions establish the six-room mansion and the player's starting state, while the room-display function presents the current location and its available information. By establishing the program's main data structures, room

configuration, and initial game state, this section provides the foundation required by the gameplay functions implemented in the later sections.

2. Humayra Mahjuba Zaman | Student ID: 2621387042

Mahjuba was responsible for the central gameplay functions in her assigned section. Her contribution includes player movement, ghost encounters, item collection, inventory display, and healing. The movement system checks available exits and required items before allowing access to restricted rooms, while ghost encounters reduce health. The item and inventory functions allow objects to be collected and managed, and the healing system allows Medicine and the Med Kit to restore health while preventing health from exceeding the maximum value. This section connects several CSE115 concepts directly to the player's interaction with the game.

3. Tilika Biswas | Student ID: 2621946042

Tilika was responsible for completing the puzzle and key system, inventory verification, main gameplay loop, input handling, Game Over condition, and winning condition. Her section contains the logic for solving the Library and Secret Chamber puzzles and generating the Silver Key and Master Key. The `hasItem()` function verifies required inventory items, while `main()` connects the previously implemented functions through the gameplay menu and switch-case selection. The final conditions determine whether the player has lost all health or has returned to the Entrance Hall with the Master Key, completing the overall game progression.

Together, the three contributions form a single integrated program: Aqif established the program structure and game state, Mahjuba implemented the main interactive gameplay systems, and Tilika completed the progression, input handling, and final game conditions. Each section therefore contributed a necessary part of the final game and allowed the group to develop and present the project as one complete system.

TESTING AND RESULTS:

Testing was carried out on the final version of *Escaping the Haunted Mansion* to check whether the main gameplay features worked as intended and whether the different parts of the program worked correctly together. The testing covered movement, item collection, ghost encounters, health recovery, puzzle solving, input handling, and the final game conditions. Both individual actions and a complete gameplay sequence were tested to identify problems during normal gameplay as well as during restricted or invalid actions.

1. Gameplay and Feature Testing

The movement system was tested by selecting both available and unavailable directions. The program prevented movement when no exit was available and restricted access to the Basement and Secret Chamber until the required items had been obtained. After the

Flashlight and Silver Key were collected, the corresponding areas became accessible. Item collection was also tested, and collected items were added to the inventory while the appropriate score was updated.

The health system was tested through the ghost encounters in the Kitchen and Basement. Entering an active ghost area reduced the player's health, while the healing system allowed Medicine and the Med Kit to restore health. The program also prevented health from increasing beyond its maximum value and removed a healing item after it was used.

The figure consists of four screenshots of a game's terminal interface, arranged in a 2x2 grid. Each screenshot shows a black background with white text.

- Top-left screenshot:** Displays the player's status: "Player: AQIF | Health: 100/100 | Score: 10".
- Top-right screenshot:** Shows the result of collecting a Med Kit: "You collected: Med Kit", "Score +10", followed by a separator line and the updated status: "Player: AQIF | Health: 84/100 | Score: 20".
- Bottom-left screenshot:** Shows a ghost encounter: "You entered the Kitchen.", "A ghost suddenly attacks you!", "You lost 16 health.", "The ghost disappears.", followed by a separator line and the updated status: "Player: AQIF | Health: 84/100 | Score: 10".
- Bottom-right screenshot:** Shows the healing process: "Healing Items:", "2. Med Kit", "Enter the item number to use: 2", "You restored 35 health.", "Current Health: 100/100", followed by a separator line and the final status: "Player: AQIF | Health: 100/100 | Score: 20".

Figure 6.1. Testing of the health and healing system.

The two puzzle systems were then tested to verify the game's progression. The correct answer to the Library puzzle generated the Silver Key, while the correct answer to the Secret Chamber puzzle generated the Master Key. Incorrect answers did not provide the required keys. Input validation was also tested with invalid menu input to ensure that the program rejected inappropriate entries and allowed gameplay to continue normally.

2. Complete Gameplay and Final Results

A complete gameplay sequence was performed to verify that the individual features operated correctly when combined. The player began in the Entrance Hall, collected the Flashlight, progressed through the mansion, solved the Library puzzle to obtain the Silver Key, accessed the Basement, obtained the required items, entered the Secret Chamber, solved the second puzzle to obtain the Master Key, and returned to the Entrance Hall. The winning message was displayed only after the required conditions had been satisfied.

```
Enter your choice: 1
You entered the Entrance Hall.

=====
CONGRATULATIONS!
=====
AQIF escaped the Haunted Mansion!
Final Score: 100
Health Left: 94
Thanks for playing!

You collected: Master Key
Score +10
Return to the Entrance Hall to escape!
```

Figure 6.2. Successful completion of the game.

The Game Over condition was also tested by allowing the player's health to reach zero. The program displayed the Game Over message and stopped the gameplay loop as expected. Overall, the testing showed that the major systems maintained the intended game state and worked together to produce the complete gameplay experience. Movement restrictions controlled progression, inventory and health changed according to player actions, puzzle solutions unlocked the required keys, and the final conditions correctly distinguished between successful completion and Game Over.

CHALLENGES AND SOLUTIONS:

The development of *Escaping the Haunted Mansion* involved several challenges because the different gameplay systems had to operate together within the scope of an introductory CSE115 project. One challenge was organizing the information associated with each room and the player. The `Room` and `Player` structures were used to keep related data together and make the program easier to manage.

Controlling the order of progression was another challenge. The program was designed to prevent access to the Basement without the Flashlight and to the Secret Chamber without the Silver Key. Health management also required several conditions because ghost attacks reduce health while Medicine and the Med Kit restore it without allowing health to exceed 100.

User input presented another difficulty because invalid entries could interfere with later `scanf()` operations. The program therefore checks whether valid input was received and clears the remaining characters when necessary. The puzzles were also connected directly to the game state: solving the Library puzzle produces the Silver Key, while solving the Secret Chamber puzzle produces the Master Key required for the final escape.

WHAT WAS LEARNED?

The project provided practical experience in applying the C programming concepts covered in CSE115 to one complete program. The group used structures, arrays, strings, functions, parameters, pointers, loops, conditional statements, switch-case selection, input/output, and random number generation as parts of the game's actual functionality.

The development process also provided experience in debugging, testing different game conditions, maintaining a shared program state, and combining separate sections of code into one working application. This helped demonstrate how individual programming concepts can work together to produce a complete interactive program.

CONCLUSION:

Escaping the Haunted Mansion successfully combines fundamental C programming concepts into an interactive text-based adventure game. The final program contains six connected rooms, item and inventory management, two puzzles, two ghost encounters, health recovery, score tracking, and both Game Over and winning conditions.

The testing process confirmed that the major gameplay systems operated as intended and that the different functions maintained a consistent game state. The project therefore achieved its purpose as a CSE115 programming exercise by applying course concepts in a practical and understandable application rather than presenting them as isolated examples.

FUTURE IMPROVEMENTS:

Future versions of the game could expand the mansion with additional rooms, puzzles, ghost types, and items to provide greater variety. Multiple endings, random events, and selectable difficulty levels could also make different playthroughs less predictable. A save-and-load system could be introduced to preserve player progress.

The program could also be divided into multiple C source files so that related functions are organized into separate modules. File handling could be used to store player progress and scores. These additions would increase the size and complexity of the project, so they were kept outside the scope of the current CSE115 implementation.